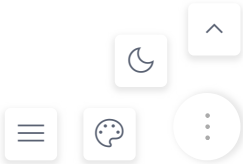


字节旗下的 AI IDE



计算几何基本模板（二维）

观前提醒：「文章仅供学习和参考，如有问题请在评论区提出」

目录

- 基本设置
- 点 + 向量
 - Point (Vector)
 - 点积（数量积、内积） (Dot)
 - 向量积，叉积 (Cross)
 - 两点间距离 (Dist)
 - 向量的模 (Len)
 - 单位向量 (Norm)
 - 两向量的夹角 (Angle)
 - 判断点在直线（向量）的哪边 (Cross)
 - 逆转角 (Rotate)
- 线
 - 直线表达式
 - Line
 - 判断三点共线 (In_one_line)
 - 点到直线的距离 (Dist_point_to_line)
 - 点到线段的距离 (Dist_point_to_seg)
 - 判断点是否在线段上 (OnSegment)
 - 判断直线与线段是否相交 (Intersect_line_seg)
 - 判断两线段是否相交 (Intersect_seg)
 - 判断两直线平行 (Line_parallel)
 - 求两直线交点 (Intersection_line)
- 多边形
 - 三角形面积 (Triangle_area)
 - 三角形四心
 - 正弦定理 & 余弦定理
 - 正多边形的一些性质和概念
 - 求多边形面积 (Polygon_area)
 - 判断点在多边形内 (InPolygon)
 - 判断凸多边形 (Is_convex)
- 圆
 - Circle
 - 扇形的面积 (SectorArea)
 - 点与圆的位置关系 (Point_with_circle)
 - 直线与圆的位置关系 (Line_with_circle)
 - 求直线与圆的交点 (Intersection_line_circle)
 - 圆与圆的位置关系 (Circle_with_circle)
 - 圆与圆的交点 (Intersection_circle_circle)
 - 求圆外一点对圆的两个切点 (TangentPo

字节旗下的 AI IDE

- 求三角形的外接圆 (get_circumcircle)
- 求三角形的内切圆 (get_incircle)
- 网格
 - 求线段上整点个数 (IntegerPoint_on_seg)
 - 求多边形上整点个数 (IntegerPoint_on_polygon)
 - 多边形内整点个数 (IntegerPoint_in_polygon)
- 极角排序
- 凸包 Andrew算法
- 最小圆覆盖问题
- 圆的面积并
- 圆与多边形的面积交
- 自适应辛普森积分
- 平面最近点对
- 参考资料

只是整理了一些基本的二维计算模板，参考资料都在最后。

每个模板都试了试具体的可行性，基本上应该没有什么错误（大概）。如果有问题，请及时联系我进行修改。

基本设置

```
long double
```

```
// 输入输出
scanf("%Lf", &x);
printf("%.6Lf\n", x);
// 函数使用: fabs1(), cos1()
```

常数定义

```
const double eps = 1e-8;    // 根据题目精度要求进行修改
const double PI = acos(-1.0); // pai, 3.1415926....

int sgn(double x) { // 进行判断，提高精度
    if (fabs(x) < eps) return 0;    // x == 0, 精度范围内的近似相等
    return x > 0 ? 1 : -1;          // 返回正负
}

int sgn(double x) { return x < -eps ? -1 : x > eps; }
```

点 + 向量

Point (Vector)

向量：点 $A(x_1, y_1), B(x_2, y_2)$ ，那么向量 $\vec{AB} = B - A = (x_2 - x_1, y_2 - y_1)$ 。

Point(x, y) 代表点 $A = (x, y)$ ，Vector(x, y) 代表向量 $\vec{AB}$ ，字节旗下的 AI IDE

```
// Need: sgn()

typedef struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {} // 构造函数, 初始值为 0

    // 重载运算符
    // 点 - 点 = 向量(向量AB = B - A)
    Point operator- (const Point &B) const { return Point(x - B.x, y - B.y); }

    // 点 + 点 = 点, 点 + 向量 = 向量, 向量 + 向量 = 向量
    Point operator+ (const Point &B) const { return Point(x + B.x, y + B.y); }

    // 向量 × 向量 (叉积)
    double operator^ (const Point &B) const { return x * B.y - y * B.x; }

    // 向量 · 向量 (点积)
    double operator* (const Point &B) const { return x * B.x + y * B.y; }

    // 点 * 数 = 点, 向量 * 数 = 向量
    Point operator* (const double &B) const { return Point(x * B, y * B); }

    // 点 / 数 = 点, 向量 / 数 = 向量
    Point operator/ (const double &B) const { return Point(x / B, y / B); }

    // 判断大小, 一般用于排序
    bool operator< (const Point &B) const { return x < B.x || (x == B.x && y < B.y); }

    // 判断相等, 点 == 点, 向量 == 向量, 一般用于判断和去重
    bool operator== (const Point &B) const { return sgn(x - B.x) == 0 && sgn(y - B.y) == 0; }

    // 判断不相等, 点 != 点, 向量 != 向量
    bool operator!= (const Point &B) const { return sgn(x - B.x) || sgn(y - B.y); }
} Vector;
```

点积 (数量积、内积) (Dot)

向量 $\vec{a}(x_1, y_1), \vec{b}(x_2, y_2)$, $\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}| \cos \theta = x_1x_2 + y_1y_2$ 。

θ 表示向量 $\vec{a}$ 旋转到向量 $\vec{b}$ 所经过的夹角。

```
// 向量 · 向量 (点积)
double operator* (Vector &A, Vector &B) { return A.x * B.x + A.y * B.y; }
```

夹角 θ 与点积大小的关系：

1. 若 $\theta = 0^\circ$, $\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}|$ 。
2. 若 $\theta = 180^\circ$, $\vec{a} \cdot \vec{b} = -|\vec{a}||\vec{b}|$ 。
3. 若 $\theta < 90^\circ$, $\vec{a} \cdot \vec{b} > 0$ 。
4. 若 $\theta = 90^\circ$, $\vec{a} \cdot \vec{b} = 0$ 。
5. 若 $\theta > 90^\circ$, $\vec{a} \cdot \vec{b} < 0$ 。

向量积, 叉积 (Cross)

字节旗下的 AI IDE

向量 $\vec{a}(x_1, y_1), \vec{b}(x_2, y_2)$, $\vec{a} \times \vec{b} = |\vec{a}||\vec{b}| \sin \theta = x_1 y_2 - x_2 y_1$ 。

θ 表示向量 $\vec{a}$ 旋转到向量 $\vec{b}$ 所经过的夹角。

// 向量 × 向量 (叉积)

```
double operator^ (Vector &A, Vector &B) { return A.x * B.y - A.y * B.x; }
```

两点间距离 (Dist)

点 $a(x_1, y_1), b(x_2, y_2)$, $ab = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

// Need: (-, *)

```
double dist(Point a, Point b) { return sqrt((a - b) * (a - b)); }
```

向量的模 (Len)

向量 $\vec{a}(x, y)$, $|\vec{a}| = \sqrt{x^2 + y^2}$ 。

// Need: (*)

```
double len(Vector A) { return sqrt(A * A); }
```

单位向量 (Norm)

// Need: (/), len()

```
Vector norm(Vector A) { return A / len(A); }
```

两向量的夹角 (Angle)

由 $\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}| \cos \theta$, 得 $\cos \theta = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}||\vec{b}|}$, 即 $\theta = \arccos \frac{\vec{a} \cdot \vec{b}}{|\vec{a}||\vec{b}|}$ 。

// Need: (*), len(), PI

```
double Angle(Vector A, Vector B) {
    double t = acos((A * B) / len(A) / len(B));
    return t; // 返回 [0, π]
    return t * (180 / PI); // 返回 [0, 180] (角度)
}
```

判断点在直线 (向量) 的哪边 (Cross)

```
// Need: (-, ^), sgn()

// 点在直线上，返回 0（三点共线）
// 点在直线的逆时针方向，返回 1
// 点在直线的顺时针方向，返回 -1

// 点 a, b（向量ab）所在的直线和点 c
// 使用的时候要注意 a 和 b 的顺序，有时顺序不同，结果不同
int Cross(Point a, Point b, Point c) { return sgn((b - a) ^ (c - a)); }

// 两种分情况使用
double Cross(Point a, Point b, Point c) { return (b - a) ^ (c - a); }
```

逆转角（Rotate）

将向量 A 逆时针旋转 $[0, \pi]$ 。

```
// Need: (*, ^)

// 向量 A 和要逆时针转的角度 [0, PI]
// PI / 2, 90度
Vector Rotate(Vector A, double b) {
    Vector B(sin(b), cos(b));
    return Vector(A ^ B, A * B);
}
```

线

直线表达式

- 一般式: $ax + by + c = 0$
- 点向式: 直线上一点 (x_0, y_0) 和方向向量 (u, v) , 有 $\frac{x-x_0}{u} = \frac{y-y_0}{v}, (u \neq 0, v \neq 0)$
- 斜截式: $y = kx + b$

Line

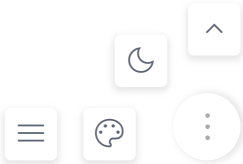
```
struct Line {
    Point s, e;
    Line() {}
    Line(Point x, Point y):s(x), e(y) {}
};
```

判断三点共线 (In_one_line)

如果三点 $A(x_1, y_1), B(x_2, y_2), C(x_3, y_3)$ 共线, 等价于 $\vec{AB} \times \vec{BC} = 0$ 。

```
// Need: sgn(), 操作符重载(-, ^)
```

字节旗下的 AI IDE



```
bool In_one_line(Point A, Point B, Point C) { return !sgn((B - A) ^ (C - B)); }
```

点到直线的距离 (Dist_point_to_line)

$\vec{a} \times \vec{b} = |\vec{a}| |\vec{b}| \sin \theta = |\vec{a}| d$, 那么 $d = \frac{\vec{a} \times \vec{b}}{|\vec{a}|}$ 。

```
// Need: (-, ^), len()
```

```
// 点 P 到直线 AB 的距离
```

```
double Dist_point_to_line(Point P, Point A, Point B) {
    Vector v1 = B - A, v2 = P - A;
    return fabs((v1 ^ v2) / len(v1));
}
```

点到线段的距离 (Dist_point_to_seg)

```
// Need: 操作符重载(==, -, *, ^), len(), sgn()
```

```
double Dist_point_to_seg(Point P, Point A, Point B) {
    if (A == B) return len(P - A); // 如果重合, 那么就是两点的距离
    Vector v1 = B - A, v2 = P - A, v3 = P - B;
    if (sgn(v1 * v2) < 0) return len(v2); // AP 最短
    if (sgn(v1 * v3) > 0) return len(v3); // BP 最短
    return fabs((v1 ^ v2) / len(v1)); // 垂线
}
```

判断点是否在线段上 (OnSegment)

```
// Need: (-, *, ^), sgn()
```

```
bool OnSegment(Point P, Point A, Point B) {
    Vector PA = A - P, PB = B - P;
    return sgn(PA ^ PB) == 0 && sgn(PA * PB) <= 0; // <=, 包括端点; <, 不包括端点
}
```

判断直线与线段是否相交 (Intersect_line_seg)

```
// Need: Cross()
```

```
// 相交, 返回 true
// 不相交, 返回 false
```

```
// 直线 ab 与线段 cd
```

```
bool Intersect_line_seg(Point a, Point b, Point c, Point d) {
    return Cross(a, b, c) * Cross(a, b, d) <= 0;
}
```

判断两线段是否相交 (Intersect_seg) 字节旗下的 AI IDE

```
// Need: Cross(), OnSegment()

// 相交, 返回 true (包括端点相交)
// 不相交, 返回 false

// 线段 ab 与线段 cd
bool Intersect_seg(Point a, Point b, Point c, Point d) {
    if (OnSegment(a, c, d) || OnSegment(b, c, d) || OnSegment(c, a, b) || OnSegment(d, a, b)) return 1;
    if (Cross(a, b, c) * Cross(a, b, d) >= 0) return 0;
    if (Cross(c, d, a) * Cross(c, d, b) >= 0) return 0;
    return 1;
}
```

判断两直线平行 (Line_parallel)

```
// Need: (-, ^), sgn()

// 返回true: 平行/重合, false: 相交
bool Line_parallel(Line A, Line B) { return sgn((A.s - A.e) ^ (B.s - B.e)) == 0; }
```

求两直线交点 (Intersection_line)

直线的两点式 (a, c) 转化为点向式 $(a, c - a)$ 。

```
// Need: (-, *D, ^)

// 首先要判断两直线是否相交, 即不平行(不重合)
// a, b 所在直线与 c, d 所在直线的交点
Point Intersection_line(Point a, Point b, Point c, Point d) {
    Vector u = b - a, v = d - c;
    double t = ((a - c) ^ v) / (v ^ u);
    return a + u * t;
}

// Need: (-, *D, ^)

Point Intersection_line(Point a, Vector u, Point b, Vector v) {
    double t = ((a - b) ^ v) / (v ^ u);
    return a + u * t;
}
```

多边形

三角形面积 (Triangle_area)

海伦公式 : $S = \sqrt{p(p-a)(p-b)(p-c)}, p = \frac{a+b+c}{2}$

```
// Need: 操作符重载(-), len()

double Triangle_area(Point A, Point B, Point C) {
```

字节旗下的 AI IDE


```
double a = len(A - B), b = len(A - C), c = len(B - C);
double p = (a + b + c) / 2;
return sqrt(p * (p - a) * (p - b) * (p - c));
}
```

已知三角形 **两边** 及 **夹角**， $S = \frac{1}{2}ab \sin C = \frac{1}{2}bc \sin A = \frac{1}{2}ac \sin B$

过原点的三角形面积为 $S_{\triangle OAB} = \frac{1}{2}|\vec{OA} \times \vec{OB}|$

那么把三角形一点移到原点（假设是 A ），那么就有 $S_{\triangle ABC} = \frac{1}{2}|\vec{AB} \times \vec{AC}|$ 。

```
// Need: (-, ^)

double Triangle_area2(Point A, Point B, Point C) {
    return fabs((B - A) ^ (C - A)) / 2;
}
```

三角形四心

- **外心**：三边中垂线的交点，到三角形三个顶点的距离相同（外接圆圆心）。
- **内心**：角平分线的交点，到三角形三边的距离相同（内切圆圆心）。
- **垂心**：三条垂线的交点。
- **重心**：三条中线的交点，到三角形三顶点距离的平方和最小的点，三角形内到三边距离之积最大的点。

三角形重心是三点各坐标的平均值 $(\frac{x_1+x_2+x_3}{3}, \frac{y_1+y_2+y_3}{3})$ 。

正弦定理 & 余弦定理

正弦定理

在三角形 $\triangle ABC$ 中，若角 A, B, C 所对应的边为 a, b, c ，则有

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R$$

其中， R 为 $\triangle ABC$ 的外接圆半径。

余弦定理

在三角形 $\triangle ABC$ 中，若角 A, B, C 所对应的边为 a, b, c ，则有

$$a^2 = b^2 + c^2 - 2bc \cos A \tag{1}$$
$$b^2 = a^2 + c^2 - 2ac \cos A \tag{2}$$
$$c^2 = a^2 + b^2 - 2ab \cos A \tag{3}$$

正多边形的一些性质和概念

内角

正 n 边形的内角和度数为： $(n - 2) \times 180^\circ$

正 n 边形的一个内角为: $\frac{(n-2) \times 180^0}{n}$

外角

正 n 边形外角和度数为: $n \times 180^0 - (n - 2) \times 180^0 = 360^0$

正 n 边形的一个外角为: $\frac{360^0}{n}$

所以正 n 边形的一个内角也可以是: $180^0 - \frac{360^0}{n}$

中心角

多边形的重心就是它所作外接圆的圆心, 所以中心角度数为: $\frac{360^0}{n}$

正 n 边形对角线数量为: $\frac{n(n-3)}{2}$

求多边形面积 (Polygon_area)

```
// Need: (-, ^)

// 因为叉积求得的三角形面积是有向的, 在外面的面积可以正负抵消掉
// 所以能够求任意多边形面积(凸, !凸)

// p[]下标从 0 开始, 长度为 n
double Polygon_area(Point *p, int n) {
    double area = 0;
    for (int i = 1; i <= n - 2; i++)
        area += (p[i] - p[0]) ^ (p[i + 1] - p[0]);
    return fabs(area / 2); // 无向面积
    return area / 2;      // 有向面积
}
```

鞋带定理 (Shoelace formula)

$$A_1(x_1, y_1), A_2(x_2, y_2), A_3(x_3, y_3), \dots, A_n(x_n, y_n) \quad (4)$$

$$S = \frac{1}{2} \left| \sum_{i=1}^n (x_i y_{i+1} - x_{i+1} y_i) \right| \quad (5)$$

```
// Need: (^)

// 原理和上面相同, 不过是把原点(0, 0) 作为被指向点

// p[] 下标从 0 开始, 长度为 n
double Polygon_area(Point *p, int n) {
    double area = 0;
    for (int i = 0, j = n - 1; i < n; j = i++)
        area += (p[j] ^ p[i]);
    return fabs(area / 2); // 无向面积
    return area / 2;      // 有向面积
}
```

判断点在多边形内 (InPolygon)

字节旗下的 AI IDE

射线法

以被测点 P 为端点，向任意方向作射线（一般水平向右作射线），统计该射线与多边形的交点数（不包括多边形顶点）。

如果为奇数，点 P 在多边形内；如果为偶数，点 P 在多边形外。

如果点 P 的纵坐标比多边形某边的纵坐标都大（都小），那么他们的交点一定在延长线上。

因为是以 $P(x_0, y_0)$ 为端点，水平向右作射线，要想判断有没有交点，所以只需要判断同 y_0 坐标下， x_0 是否在 x 的左边。

$$k = \frac{y_1 - y_2}{x_1 - x_2} \quad (6)$$

$$\text{代入}(x_1, y_1), \text{得 } b = y_1 - \frac{y_1 - y_2}{x_1 - x_2} x_1 \quad (7)$$

$$y = \frac{y_1 - y_2}{x_1 - x_2} x - \frac{y_1 - y_2}{x_1 - x_2} x_1 + y_1 \quad (8)$$

$$\text{代入} y_0, \text{得 } x = (y_0 - y_1) \frac{x_1 - x_2}{y_1 - y_2} + x_1 \quad (9)$$

// Need: sgn(), OnSegment()

// 适用于任意多边形，不用考虑精度误差和多边形的给出顺序

// 点在多边形边上，返回 -1

// 点在多边形内，返回 1

// 点在多边形外，返回 0

// p[] 的下标从 0 开始，长度为 n

```
int InPolygon(Point P, Point *p, int n) {
    bool flag = false;    // 相当于计数
    for (int i = 0, j = n - 1; i < n; j = i++) {
        Point p1 = p[i], p2 = p[j];
        if (OnSegment(P, p1, p2)) return -1;
        if (sgn(P.y - p1.y) > 0 == sgn(P.y - p2.y) > 0) continue;
        if (sgn((P.y - p1.y) * (p1.x - p2.x) / (p1.y - p2.y) + p1.x - P.x) > 0)
            flag = !flag;
    }
    return flag;
}
```

根据方向判断

局限性：只能用于判断点是否在 **凸多边形** 内，并且要求点的给出顺序是顺时针（或逆时针）并且多边形点的个数 $n > 2$ 。

// Need: Cross()

// 点在多边形边上，返回 -1

// 点在多边形内，返回 1

// 点在多边形外，返回 0

// p[] 的下标从 0 开始，长度为 n

```
int InPolygon(Point P, Point *p, int n) {
    int st[3] = {0, 0, 0};
    for (int i = 0, j = n - 1; i < n; j = i++) {
        st[Cross(p[j], p[i], P) + 1]++;
        if (st[0] && st[2]) return 0;
    }
    if (st[1]) return -1;
    return 1;
}
```

字节旗下的 AI IDE

判断凸多边形 (Is_convex)

```
// Need: (-, ^), sgn()

// 顶点必须按顺时针(或逆时针)给出, 允许共线边
// p[] 下标从 0 开始, 长度为 n
bool Is_convex(Point *p, int n) {
    bool s[3] = {0, 0, 0};
    for (int i = 0, j = n - 1, k = n - 2; i < n; k = j, j = i++) {
        int cnt = sgn((p[i] - p[j]) ^ (p[k] - p[j])) + 1;
        s[cnt] = true;
        if (s[0] && s[2]) return false;
    }
    return true;
}
```

圆

Circle

弧长公式: $L = \alpha \times r$, 弧长 = 半径 $\times$ 圆心角。

```
// Need: Point()

struct Circle {
    Point o;
    double r;
    Circle(Point _o = Point(), double _r = 0) : o(_o), r(_r) {}

    // 圆的面积
    double Circle_S() { return PI * r * r; }
    // 圆的周长
    double circle_C() { return 2 * PI * r; }
};
```

扇形的面积 (SectorArea)

设扇形的半径为 r , 弧长为 l , 面积为 S , 圆心角为 α , 那么有

$$S = \frac{1}{2}lr = \frac{1}{2}\alpha r^2$$

```
// Need: (^), Angle(), sgn()

// 扇形的两交点A, B 和圆的半径 R
double SectorArea(Point A, Point B, double R) {
    double angle = Angle(A, B);
    if (sgn(A ^ B) < 0) angle = -angle;
    return R * R * angle / 2;
}
```

字节旗下的 AI IDE

点与圆的位置关系 (Point_with_circle)

```
// Need: sgn(), dist()

// 点在圆上, 返回 0
// 点在圆外, 返回 -1
// 点在圆内, 返回 1

int Point_with_circle(Point p, Circle c) {
    double d = dist(p, c.o);
    if (sgn(d - c.r) == 0) return 0;
    if (sgn(d - c.r) > 0) return -1;
    return 1;
}
```

直线与圆的位置关系 (Line_with_circle)

```
// Need: sgn(), Dist_point_to_line()

// 相切, 返回 0
// 相交, 返回 1
// 相离, 返回 -1

int Line_with_circle(Point A, Point B, Circle c) {
    double d = Dist_point_to_line(c.o, A, B);
    if (sgn(d - c.r) == 0) return 0;
    if (sgn(d - c.r) > 0) return -1;
    return 1;
}
```

求直线与圆的交点 (Intersection_line_circle)

```
// Need: (-, +, *P, *D, /), sgn()

// 直线与圆相交, 返回两点
// 直线与圆相切, 返回两个一样的相切点

pair<Point, Point> Intersection_line_circle(Point A, Point B, Circle c) {
    Vector AB = B - A;
    Vector pr = A + AB * ((c.o - A) * AB / (AB * AB));
    double base = sqrt(c.r * c.r - ((pr - c.o) * (pr - c.o)));

    if (sgn(base) == 0) return make_pair(pr, pr);

    Vector e = AB / sqrt(AB * AB);
    return make_pair(pr + e * base, pr - e * base);
}
```

圆与圆的位置关系 (Circle_with_circle)

```
// Need: dist()

// 相离, 返回 -1
```

字节旗下的 AI IDE

```

// 外切, 返回 0
// 内切(A 包含 B), 返回 1
// 内切(B 包含 A), 返回 2
// 内含(A 包含 B), 返回 3
// 内含(B 包含 A), 返回 4
// 相交, 返回 5

int Circle_with_circle(Circle A, Circle B) {
    double len1 = dist(A.o, B.o);
    double len2 = A.r + B.r;
    if (sgn(len1 - len2) > 0) return -1;
    if (sgn(len1 - len2) == 0) return 0;
    if (sgn(len1 + len2 - 2 * A.r) == 0) return 1;
    if (sgn(len1 + len2 - 2 * B.r) == 0) return 2;
    if (sgn(len1 + len2 - 2 * A.r) < 0) return 3;
    if (sgn(len1 + len2 - 2 * B.r) < 0) return 4;
    return 5;
}

```

圆与圆的交点 (Intersection_circle_circle)

```

// Need: (-, +), len()

// 相交, 返回两点坐标
// 相切, 返回两个一样的相切点

// 要先判断是否相交或相切再调用
pair<Point, Point> Intersection_circle_circle(Circle A, Circle B) {
    Vector AB = B.o - A.o;
    double d = len(AB);
    double a = acos((A.r * A.r + d * d - B.r * B.r) / (2.0 * A.r * d));
    double t = atan2(AB.y, AB.x);
    Vector x(A.r * cos(t + a), A.r * sin(t + a));
    Vector y(A.r * cos(t - a), A.r * sin(t - a));
    return make_pair(A.o + x, A.o + y);
}

```

求圆外一点对圆的两个切点 (TangentPoint_point_circle)

```

// Need: (-, *, ^, +), dist()

// 返回两个切点坐标

// 一定要先判断点在圆外, 然后再调用
pair<Point, Point> TangentPoint_point_circle(Point p, Circle c) {
    double d = dist(p, c.o);
    double l = sqrt(d * d - c.r * c.r);
    Vector e = (c.o - p) / d;
    double angle = asin(c.r / d);

    Vector t1(sin(angle), cos(angle));
    Vector t2(sin(-angle), cos(-angle));
    Vector e1(e ^ t1, e * t1);
    Vector e2(e ^ t2, e * t2);
    e1 = e1 * l + p;
    e2 = e2 * l + p;
}

```

字节旗下的 AI IDE

```

    return make_pair(e1, e2);
}

```

求三角形的外接圆 (get_circumcircle)

已知 $\triangle ABC$ 的三个顶点 $A(x_1, y_1), B(x_2, y_2), C(x_3, y_3)$ ，假设其外接圆圆心为 $O(x_0, y_0)$ ，半径为 r 。

根据外接圆圆心的特点，圆心到三点的距离相等，有

$$(x_1 - x_0)^2 + (y_1 - y_0)^2 = (x_2 - x_0)^2 + (y_2 - y_0)^2 = (x_3 - x_0)^2 + (y_3 - y_0)^2$$

推导出，

$$x_0 = \frac{(y_2 - y_1)(y_3^2 - y_1^2 + x_3^2 - x_1^2) - (y_3 - y_1)(y_2^2 - y_1^2 + x_2^2 - x_1^2)}{2(x_3 - x_1)(y_2 - y_1) - 2(x_2 - x_1)(y_3 - y_1)} \quad (10)$$

$$y_0 = \frac{(x_2 - x_1)(x_3^2 - x_1^2 + y_3^2 - y_1^2) - (x_3 - x_1)(x_2^2 - x_1^2 + y_2^2 - y_1^2)}{2(y_3 - y_1)(x_2 - x_1) - 2(y_2 - y_1)(x_3 - x_1)} \quad (11)$$

$$r = \sqrt{(x_1 - x_0)^2 + (y_1 - y_0)^2} \quad (12)$$

```
// Need: dist()
```

```

Circle get_circumcircle(Point A, Point B, Point C) {
    double Bx = B.x - A.x, By = B.y - A.y;
    double Cx = C.x - A.x, Cy = C.y - A.y;
    double D = 2 * (Bx * Cy - By * Cx);

    double x = (Cy * (Bx * Bx + By * By) - By * (Cx * Cx + Cy * Cy)) / D + A.x;
    double y = (Bx * (Cx * Cx + Cy * Cy) - Cx * (Bx * Bx + By * By)) / D + A.y;
    Point P(x, y);
    return Circle(P, dist(A, P));
}

```

求三角形的内切圆 (get_incircle)

```
// Need: (*D, /), dist(), Dist_point_to_line()
```

```

Circle get_incircle(Point A, Point B, Point C) {
    double a = dist(B, C);
    double b = dist(A, C);
    double c = dist(A, B);
    Point p = (A * a + B * b + C * c) / (a + b + c);
    return Circle(p, Dist_point_to_line(p, A, B));
}

```

网格

求线段上整点个数 (IntegerPoint_on_seg)

```
// 要保证传入的点是整点
```

```

int IntegerPoint_on_seg(Point A, Point
    int x = abs(A.x - B.x);

```

字节旗下的 AI IDE

```
int y = abs(A.y - B.y);
if (x == 0 || y == 0) return 1;
return __gcd(x, y) + 1; // 包含端点
return __gcd(x, y) - 1; // 不包含端点
}
```

求多边形上整点个数 (IntegerPoint_on_polygon)

```
// 返回多边形边上整点的个数
// 点需要是顺时针(逆时针)给出

// p[] 下标从 0 开始, 长度为 n
int IntegerPoint_on_polygon(Point *p, int n) {
    int res = 0;
    for (int i = 0, j = n - 1; i < n; j = i++) {
        int x = abs(p[i].x - p[j].x);
        int y = abs(p[i].y - p[j].y);
        res += __gcd(x, y);
    }
    return res;
}
```

多边形内整点个数 (IntegerPoint_in_polygon)

皮克定理 (Pick 's Theorem)

任一顶点在网格上的封闭多边形, 面积为 A , 内部格点数为 I , 边界上格点数为 B 。那么有

$$A = I + \frac{1}{2}B - 1$$

```
// Need: Polygon_area(), IntegerPoint_on_polygon()

// 返回不包括边界的, 多边形内整点个数
int IntegerPoint_in_polygon(Point *p, int n) {
    double A = Polygon_area(p, n);
    double B = IntegerPoint_on_polygon(p, n);
    return A - B / 2 + 1;
}
```

极角排序

```
// Need: (-, ^), len(), sgn()

// 排序常数大, 但精度高

Point p[N]; // 要排序的点
Point o(0, 0); // 极点自定义

// 获取象限 (0, 1, 2, 3)
int Quadrant(Vector p) { return sgn(p.y < 0) << 1 | sgn(p.x < 0) ^ sgn(p.y < 0); }

// 比较函数
bool cmp(Point a, Point b) {
```

字节旗下的 AI IDE


```

Vector p = a - o, q = b - o;
int x = Quadrant(p), y = Quadrant(q);
if (x == y) {
    if (sgn(p ^ q) == 0) return len(p) < len(q);
    return sgn(p ^ q) > 0;
}
return x < y;
}

```

凸包 Andrew算法

给定平面上的点集，**凸包**就是将最外层的点连接起来构成的凸多边形，它能包含点集中的所有点。

Andrew算法： $O(n\log n)$

```

// Need: (<), Cross()

Point s[N]; // 用来存凸包多边形的顶点
int top = 0;

// 点集 p[] 的下标从 1 开始，长度为 n
void Andrew(Point *p, int n) {
    sort(p + 1, p + n + 1);
    for (int i = 1; i <= n; i++) { // 下凸包
        while (top > 1 && Cross(s[top - 1], s[top], p[i]) <= 0) top--;
        s[++top] = p[i];
    }
    int t = top;
    for (int i = n - 1; i >= 1; i--) { // 上凸包
        while (top > t && Cross(s[top - 1], s[top], p[i]) <= 0) top--;
        s[++top] = p[i];
    }

    top--; // 因为首尾都会加一次第一个点，所以去掉最后一个
}

```

最小圆覆盖问题

在一个平面上有 n 个点，求一个半径最小的圆，能覆盖所有的点。

随机增量法： $O(n)$

```

// Need: (+, /), sgn(), dist(), get_circumcircle()

// p[] 下标从 0 开始，长度为 n
Circle get_min_circle(Point *p, int n) {
    // 随机化，防止被卡
    for (int i = 0; i < n; i++) swap(p[rand() % n], p[rand() % n]);

    Point o = p[0];
    double r = 0;

    for (int i = 0; i < n; i++) {

```

字节旗下的 AI IDE

```

    if (sgn(dist(o, p[i]) - r) <= 0) continue;
    o = (p[i] + p[0]) / 2;
    r = dist(p[i], p[0]) / 2;
    for (int j = 1; j < i; j++) {
        if (sgn(dist(o, p[j]) - r) <= 0) continue;
        o = (p[i] + p[j]) / 2;
        r = dist(p[i], p[j]) / 2;
        for (int k = 0; k < j; k++) {
            if (sgn(dist(o, p[k]) - r) <= 0) continue;
            o = get_circumcircle(p[i], p[j], p[k]).o;
            r = dist(o, p[i]);
        }
    }
}
return Circle(o, r);
}

```

圆的面积并

求平面上 n 个圆的面积并。

格林公式： $n^2 \log n$

```

#include <bits/stdc++.h>
using namespace std;

const int N = 1010;
const double eps = 1e-8;
const double PI = acos(-1.0);
inline int sgn(double x) { return x < -eps ? -1 : x > eps; }

typedef struct Point {
    double x, y;
    Point(double xx = 0, double yy = 0): x(xx), y(yy){}
    Point operator-(const Point &B) const { return Point(x - B.x, y - B.y); }
    double operator*(const Point &B) const { return x * B.x + y * B.y; }
    bool operator< (const Point &B) const { return x < B.x || (x == B.x && y < B.y); }
    bool operator==(const Point &B) const { return !sgn(x - B.x) && !sgn(y - B.y); }
} Vector;
double dist(Point a, Point b) { return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y)); }

struct Circle {
    Point o;
    double r;
    Circle(Point oo = Point(), double rr = 0): o(oo), r(rr){}
    bool operator<(const Circle &B) const { return o < B.o || o == B.o && r < B.r; }
    bool operator==(const Circle &B) const { return o == B.o && !sgn(r - B.r); }
};

Circle c[N];
pair<double, int> st[N << 1];
int n;

double cal(int x) {
    int top = 0, cnt = 0;
    for (int i = 1; i <= n; i++) {
        if (i == x) continue;

```

字节旗下的 AI IDE

```

    double dis = dist(c[i].o, c[x].o);
    double r1 = c[x].r, r2 = c[i].r;
    if (sgn(r1 + dis - r2) <= 0) return 0.0;
    if (sgn(r2 + dis - r1) <= 0 || sgn(r2 + r1 - dis) <= 0) continue;
    double del = acos((r1 * r1 + dis * dis - r2 * r2) / (2 * r1 * dis));
    double angle = atan2(c[i].o.y - c[x].o.y, c[i].o.x - c[x].o.x);
    double l = angle - del, r = angle + del;
    if (sgn(l + PI) < 0) l += 2 * PI;
    if (sgn(r - PI) >= 0) r -= 2 * PI;
    if (sgn(l - r) > 0) cnt++;
    st[++top] = {l, 1}, st[++top] = {r, -1};
}

st[0] = {-PI, 0}, st[++top] = {PI, 0};
sort(st + 1, st + top + 1);
double res = 0;
for (int i = 1; i <= top; cnt += st[i++].second) {
    if (cnt) continue;
    Point o = c[x].o;
    double r = c[x].r, t1 = st[i - 1].first, t2 = st[i].first;
    res += r * (r * (t2 - t1) + o.x * (sin(t2) - sin(t1)) - o.y * (cos(t2) - cos(t1)));
}
return res;
}

double get_area() {
    sort(c + 1, c + n + 1);
    n = unique(c + 1, c + n + 1) - c - 1;
    double ans = 0;
    for (int i = 1; i <= n; i++) ans += cal(i);
    return ans / 2;
}

int main() {
    cin >> n;
    for (int i = 1; i <= n; i++) scanf("%lf%lf%lf", &c[i].o.x, &c[i].o.y, &c[i].r);
    printf("%.3lf\n", get_area());

    return 0;
}

```

圆与多边形的面积交

给定一个多边形和圆，求多边形和圆的面积交。

三角剖分： $O(nT)$

// Need: (-, +, *D, *, ^, /), sgn(), Intersection_line(点向量版), OnSegment(), Rotate()
 // SectorArea(), Angle(), norm(), len(), dist(),

// 返回圆点到 ab 线段的距离，并带回圆与线段的交点 pa, pb

```

double getDP2(Point a, Point b, Circle c, Point &pa, Point &pb) {
    Point o = c.o;
    double R = c.r;
    Point e = Intersection_line(a, b -
    double d = dist(o, e);
    if (!OnSegment(e, a, b)) d = min(dist(o, a), dist(o, b));
}

```

字节旗下的 AI IDE

; // 垂足点

```

    if (R <= d) return d;
    double Len = sqrt(R * R - dist(o, e) * dist(o, e));
    pa = e + norm(a - b) * Len;
    pb = e + norm(b - a) * Len;
    return d;
}

double getArea(Point a, Point b, Circle C) {    // 面积之交
    Point o = C.o;
    double R = C.r;
    if (sgn(a ^ b) == 0) return 0;    // 共线
    double da = dist(o, a), db = dist(o, b);
    if (sgn(R - da) >= 0 && sgn(R - db) >= 0) return (a ^ b) / 2;    // ab 在圆内
    Point pa, pb;
    double d = getDP2(a, b, C, pa, pb);
    if (sgn(R - d) <= 0) return SectorArea(a, b, R);    // ab 在圆外
    if (sgn(R - da) >= 0) return (a ^ pb) / 2 + SectorArea(pb, b, R);    // a 在圆内
    if (sgn(R - db) >= 0) return SectorArea(a, pa, R) + (pa ^ b) / 2;    // b 在圆内
    return SectorArea(a, pa, R) + (pa ^ pb) / 2 + SectorArea(pb, b, R);    // ab 是割线
}

// 返回所求的面积交
double Intersection_Area(Point *p, int n, Circle C) {
    // 平移
    for (int i = 0; i < n; i++) p[i] = p[i] - C.o;
    C.o = Point(0.0, 0.0);

    double area = 0;
    for (int i = 0, j = n - 1; i < n; j = i++) area += getArea(p[j], p[i], C);
    return fabs(area);
}

// 调用
Point p[N];
int n;
Circle C;

double area = Intersection_Area(p, n, C);

```

自适应辛普森积分

辛普森公式

对二次函数 $f(x) = ax^2 + bx + c$ 在区间 $[l, r]$ 上求定积分（即图像与 x 轴围成的面积）

$$\text{原函数: } F(x) = \frac{a}{3}x^3 + \frac{b}{2}x^2 + cx + d \quad (13)$$

$$\int_l^r f(x)dx = \frac{a}{3}(r^3 - l^3) + \frac{b}{2}(r^2 - l^2) + c(r - l) \quad (14)$$

$$= (r - l) \left[\frac{a}{3}(r^2 - l^2) + \frac{b}{2}(r - l) + c \right] \quad (15)$$

$$= \frac{r - l}{6} (2al^2 + 2ar^2 + 2alr + 3bl + 3br + 6c) \quad (16)$$

$$= \frac{r - l}{6} [(al^2 + bl + c) + (ar^2 + br + c) + 4(a(\frac{l+r}{2})^2 + b(\frac{l+r}{2}) + c)] \quad (17)$$

$$= \frac{(r - l)(f(l) + f(r) + 4f(\frac{l+r}{2}))}{6} \quad (18)$$

字节旗下的 AI IDE

即 $\int_l^r f(x)dx = \frac{(r-l)(f(l)+f(r)+4f(\frac{l+r}{2}))}{6}$ （即区间宽度 $\times$ 均值高度）。

自适应辛普森法： $O(\log \frac{n}{\epsilon})$

把任意**可积分函数**分割成很多段，对积分区间不断二分，每次判断当前段和二次函数的相似程度，如果足够相似就直接代入辛普森公式计算，否则就继续分割递归求解。

```
// Need: sgn()

// 积分函数，是啥填啥
double f(double x) {
    // ...
}

// 辛普森公式
double simpson(double l, double r) {
    return (r - l) * (f(l) + f(r) + 4 * f((l + r) / 2)) / 6;
}

// 自适应
double asr(double l, double r, double ans) {
    double mid = (l + r) / 2, a = simpson(l, mid), b = simpson(mid, r);
    if (sgn(a + b - ans) == 0) return ans;
    return asr(l, mid, a) + asr(mid, r, b);
}

// 调用
double ans = asr(l, r, 0);
```

平面最近点对

给定平面上 n 个点，求最近点对之间的距离。

分治算法 + 归并排序： $O(n \log n)$

```
// Need: dist()

Point p[N], t[N]; // p[] 存点，t[] 是辅助数组

double divide(int l, int r) {
    double d = 2e9;
    if (l == r) return d;
    int mid = l + r >> 1;
    Point tmp = p[mid];
    d = min(divide(l, mid), divide(mid + 1, r)); // 分治

    // 归并排序部分
    int k = 0, i = l, j = mid + 1, tt = 0;
    while (i <= mid && j <= r)
        if (p[i].y < p[j].y) t[k++] = p[i++];
        else t[k++] = p[j++];
    while (i <= mid) t[k++] = p[i++];
    while (j <= r) t[k++] = p[j++];
    for (i = l, j = 0; i <= r; i++, j++) p[i] = t[j];

    for (int i = 0; i < k; i++)
        if (fabs(tmp.x - t[i].x) < d) t[tt++] = t[i];
```

字节旗下的 AI IDE

```
for (int i = 0; i < tt; i++)
    for (int j = i + 1; j < tt && t[j].y - t[i].y < d; j++)
        d = min(d, dist(t[i], t[j]));
return d;
}

// 调用
int n; // 点的个数
double dis = divide(1, n);
```

参考资料

计算几何模板整理 - 知乎

计算几何的模板（大神整理）_计算几何模板_clasky的博客-CSDN博客

详谈判断点在多边形内的七种方法（最全面）WilliamSun0122的博客-CSDN博客

二维计算几何基础 - OI Wiki

计算几何之求圆与直线的交点 - 知乎

计算几何之圆与圆的交点 - 知乎

求圆外一点做圆切线的切点坐标（算法）_圆外一点引两条切线,切点的坐标_zdpBuilder的博客-CSDN博客

随机增量法 - OI Wiki

计算三角形的外接圆 - 知乎

570 向量运算 点线关系【计算几何】_哔哩哔哩

571 叉积应用 线线关系【计算几何】_哔哩哔哩

572 三角剖分 面积计算【计算几何】_哔哩哔哩

579 自适应辛普森积分【计算几何】_哔哩哔哩

577 平面最近点对 分治算法【计算几何】_哔哩哔哩

577 平面最近点对 分治算法 - 董晓 - 博客园

圆的面积并（格林公式） - bztMinamoto - 博客园

作者：Oneway

出处：<https://www.cnblogs.com/oneway10101/p/17642080.html>

版权：本作品采用「署名-非商业性使用-相同方式共享 4.0 国际」许可协议进行许可。

分类：学习笔记

标签：计算几何，凸包，极角排序，最小圆覆盖，三角剖分

5

0

« 上一篇：欧拉定理 & 扩展欧拉定理

» 下一篇：关于我突然想写个简单的退役小作文这件事

字节旗下的 AI IDE 2023-08-19 09:29 Oneway 56 (1) 举报

登录后才能查看或发表评论，立即 登录 或者 逛逛 博客园首页

- 【推荐】注册飞算 JavaAI 开发助手，立得京东e卡！分享体验再领30元
- 【推荐】100%开源！大型工业跨平台软件C++源码提供，建模，组态！
- 【推荐】AI 的力量，开发者的翅膀：欢迎使用 AI 原生开发工具 TRAE
- 【推荐】2025 HarmonyOS 鸿蒙创新赛正式启动，百万大奖等你挑战

博客园 © 2004-2025

 浙公网安备 33010602011771号 浙ICP备2021040463号-3 & Theme Silence v3.0.0-rc2

字节旗下的 AI IDE

